

BFS 与 DFS 搜索算法完全指南

算法学习笔记

2026 年 5 月 13 日

目录

1	搜索算法基础概念	2
1.1	什么是搜索算法？	2
1.2	两种核心搜索策略	2
1.3	时间复杂度与空间复杂度对比	2
2	深度优先搜索（DFS）	3
2.1	DFS 的核心思想	3
2.2	DFS 的两种实现方式	3
2.2.1	递归实现（最常用）	3
2.2.2	迭代实现（使用栈）	4
2.3	DFS 的应用场景	6
2.4	DFS 经典问题：全排列	6
3	广度优先搜索（BFS）	7
3.1	BFS 的核心思想	7
3.2	BFS 的标准模板	8
3.3	BFS 的应用场景	10
3.4	BFS 经典问题：迷宫最短路径	10
4	DFS 与 BFS 的选择策略	12
5	经典例题详解	12
5.1	例题 1：岛屿数量（基础 DFS/BFS）	12
5.2	例题 2：岛屿数量 - BFS 解法	14
5.3	例题 3：单词接龙（BFS 求最短路径）	15
5.4	例题 4：课程表（拓扑排序 - DFS）	17
5.5	例题 5：课程表 - BFS 拓扑排序	19

6	进阶题目详解	20
6.1	进阶题 1: 被围绕的区域 (DFS/BFS)	20
6.2	进阶题 2: 扫雷游戏 (BFS + 模拟)	22
6.3	进阶题 3: 太平洋大西洋水流问题	25
6.4	进阶题 4: 最短路径访问所有节点 (状态压缩 BFS)	27
6.5	进阶题 5: 01 矩阵 (多源 BFS)	29
7	搜索算法的优化技巧	31
7.1	剪枝优化	31
7.2	双向 BFS	33
7.3	记忆化搜索 (Memoization)	34
8	总结与对比	36
9	练习题推荐	37

1 搜索算法基础概念

1.1 什么是搜索算法？

搜索算法是计算机科学中最基础、最重要的算法之一。它用于在数据集合中查找特定信息或遍历整个数据结构。

搜索算法的本质

搜索算法的核心是状态空间探索：

- 每个问题可以看作一个状态图
- 图中的节点代表状态
- 图中的边代表状态转移
- 搜索算法就是在这个图中找到从初始状态到目标状态的路径

1.2 两种核心搜索策略

DFS vs BFS 形象对比

- DFS（深度优先搜索）：像走迷宫，一条路走到黑，走不通再回头
 - 比喻：一个人探索迷宫，一直往前走直到无路可走，然后返回上一个岔路口
- BFS（广度优先搜索）：像水波扩散，一层一层向外扩展
 - 比喻：一群人同时从起点出发，每一步所有的人都向外走一步

1.3 时间复杂度与空间复杂度对比

算法	时间复杂度	空间复杂度	数据结构
DFS	$O(V + E)$	$O(h)$ 递归栈深度	栈 (Stack)
BFS	$O(V + E)$	$O(w)$ 最大层宽度	队列 (Queue)

其中 V 是顶点数， E 是边数， h 是最大深度， w 是最大宽度。

2 深度优先搜索（DFS）

2.1 DFS 的核心思想

DFS 遵循”深入优先”的原则，尽可能深地搜索树或图的分支。

DFS 的三个关键步骤

1. 选择：从当前节点选择一个未访问的邻接节点
2. 深入：移动到该节点，继续深入探索
3. 回溯：当无路可走时，返回上一个节点，尝试其他分支

2.2 DFS 的两种实现方式

2.2.1 递归实现（最常用）

递归实现 DFS 是最自然的方式，利用系统栈来保存状态。

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      const int MAXN = 100005;
6      vector<int> graph[MAXN]; // 邻接表存图
7      bool visited[MAXN];      // 标记节点是否访问过
8
9      // DFS递归实现
10     void dfs(int u) {
11         // 1. 处理当前节点
12         visited[u] = true;
13         cout << "访问节点: " << u << endl;
14
15         // 2. 遍历所有邻接节点
16         for (int v : graph[u]) {
17             // 3. 如果未访问，深入探索
18             if (!visited[v]) {
19                 dfs(v); // 递归深入
20             }
21         }
22         // 4. 回溯（自动发生，递归返回即回溯）
```

```

23     }
24
25     // 使用示例
26     int main() {
27         int n, m;    // n个节点, m条边
28         cin >> n >> m;
29
30         // 建图
31         for (int i = 0; i < m; i++) {
32             int u, v;
33             cin >> u >> v;
34             graph[u].push_back(v);
35             graph[v].push_back(u);    // 无向图
36         }
37
38         // 从节点1开始DFS
39         dfs(1);
40
41         return 0;
42     }

```

Listing 1: DFS 递归实现 - 图的遍历

2.2.2 迭代实现（使用栈）

使用显式栈来模拟递归过程，避免递归深度过大导致栈溢出。

```

1     #include <iostream>
2     #include <vector>
3     #include <stack>
4     using namespace std;
5
6     const int MAXN = 100005;
7     vector<int> graph[MAXN];
8     bool visited[MAXN];
9
10    // DFS迭代实现
11    void dfs_iterative(int start) {
12        stack<int> stk;
13        stk.push(start);
14

```

```

15     while (!stk.empty()) {
16         int u = stk.top();
17         stk.pop();
18
19         // 可能在栈中有重复节点，需要再次检查
20         if (visited[u]) continue;
21
22         visited[u] = true;
23         cout << "访问节点: " << u << endl;
24
25         // 将所有未访问的邻接节点入栈
26         for (int v : graph[u]) {
27             if (!visited[v]) {
28                 stk.push(v);
29             }
30         }
31     }
32 }
33
34 int main() {
35     int n = 5;
36     graph[1] = {2, 3};
37     graph[2] = {1, 4, 5};
38     graph[3] = {1};
39     graph[4] = {2};
40     graph[5] = {2};
41
42     dfs_iterative(1);
43
44     return 0;
45 }

```

Listing 2: DFS 迭代实现 - 使用栈

2.3 DFS 的应用场景

DFS 的典型应用

1. 连通性问题：判断图的连通分量数量
2. 路径搜索：查找从起点到终点的所有路径
3. 拓扑排序：有向无环图的线性排序
4. 环检测：判断图中是否存在环
5. 回溯算法：排列、组合、子集等问题
6. 树的遍历：前序、中序、后序遍历

2.4 DFS 经典问题：全排列

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      vector<int> path;           // 当前路径（当前排列）
6      vector<bool> used;         // 标记数字是否使用过
7      vector<vector<int>> result; // 存储所有结果
8
9      // 生成1~n的所有排列
10     void backtrack(int n) {
11         // 终止条件：路径长度等于n，得到一个完整排列
12         if (path.size() == n) {
13             result.push_back(path);
14             // 打印排列
15             for (int num : path) {
16                 cout << num << " ";
17             }
18             cout << endl;
19             return;
20         }
21
22         // 尝试每个数字
23         for (int i = 1; i <= n; i++) {
24             if (!used[i]) {
```

```

25         // 选择
26         used[i] = true;
27         path.push_back(i);
28
29         // 深入
30         backtrack(n);
31
32         // 回溯（撤销选择）
33         path.pop_back();
34         used[i] = false;
35     }
36 }
37
38
39 int main() {
40     int n = 3;
41     used.resize(n + 1, false);
42
43     cout << "1~3的所有排列:" << endl;
44     backtrack(n);
45
46     return 0;
47 }
48 /* 输出:
49 1 2 3
50 1 3 2
51 2 1 3
52 2 3 1
53 3 1 2
54 3 2 1
55 */

```

Listing 3: DFS 回溯 - 全排列问题

3 广度优先搜索（BFS）

3.1 BFS 的核心思想

BFS 遵循”广度优先”的原则，逐层访问节点。

BFS 的关键特征

1. 层次性：按照距离起点的层数逐层访问
2. 最短路径：在无权图中能找到最短路径
3. 队列实现：使用队列保证先进先出的顺序
4. 非递归：天然适合迭代实现

3.2 BFS 的标准模板

```
1      #include <iostream>
2      #include <vector>
3      #include <queue>
4      using namespace std;
5
6      const int MAXN = 100005;
7      vector<int> graph[MAXN];
8      bool visited[MAXN];
9      int dist[MAXN]; // 记录从起点到每个节点的距离
10
11     // BFS 实现
12     void bfs(int start) {
13         queue<int> q;
14
15         // 1. 将起点入队
16         q.push(start);
17         visited[start] = true;
18         dist[start] = 0;
19
20         // 2. 开始 BFS
21         while (!q.empty()) {
22             int u = q.front();
23             q.pop();
24
25             cout << "访问节点: " << u
26                  << ", 距离: " << dist[u] << endl;
27
28             // 3. 遍历所有邻接节点
```

```

29         for (int v : graph[u]) {
30             if (!visited[v]) {
31                 // 4. 未访问的节点入队
32                 visited[v] = true;
33                 dist[v] = dist[u] + 1; // 更新距离
34                 q.push(v);
35             }
36         }
37     }
38 }
39
40 int main() {
41     int n = 5;
42     graph[1] = {2, 3};
43     graph[2] = {1, 4, 5};
44     graph[3] = {1};
45     graph[4] = {2};
46     graph[5] = {2};
47
48     cout << "BFS从节点1开始:" << endl;
49     bfs(1);
50
51     return 0;
52 }
53 /* 输出:
54 访问节点: 1, 距离: 0
55 访问节点: 2, 距离: 1
56 访问节点: 3, 距离: 1
57 访问节点: 4, 距离: 2
58 访问节点: 5, 距离: 2
59 */

```

Listing 4: BFS 标准模板 - 图的层序遍历

3.3 BFS 的应用场景

BFS 的典型应用

1. 最短路径：无权图中求最短路径
2. 层次遍历：树的层序遍历
3. 连通分量：统计连通块数量
4. 迷宫问题：找最短逃生路径
5. 单词接龙：词语变换的最短序列
6. 社交网络：计算好友关系的度数

3.4 BFS 经典问题：迷宫最短路径

```
1      #include <iostream>
2      #include <vector>
3      #include <queue>
4      using namespace std;
5
6      struct Node {
7          int x, y;          // 坐标
8          int steps;        // 从起点到这里的步数
9      };
10
11     // 四个方向：上、下、左、右
12     int dx[] = {-1, 1, 0, 0};
13     int dy[] = {0, 0, -1, 1};
14
15     int shortestPath(vector<vector<int>>& maze,
16                     int startX, int startY,
17                     int endX, int endY) {
18         int n = maze.size();
19         int m = maze[0].size();
20
21         vector<vector<bool>> visited(n, vector<bool>(m, false));
22         queue<Node> q;
23
24         // 起点入队
```

```

25     q.push({startX, startY, 0});
26     visited[startX][startY] = true;
27
28     while (!q.empty()) {
29         Node cur = q.front();
30         q.pop();
31
32         // 到达终点
33         if (cur.x == endX && cur.y == endY) {
34             return cur.steps;
35         }
36
37         // 尝试四个方向
38         for (int i = 0; i < 4; i++) {
39             int nx = cur.x + dx[i];
40             int ny = cur.y + dy[i];
41
42             // 检查边界和障碍
43             if (nx >= 0 && nx < n && ny >= 0 && ny < m
44                 && maze[nx][ny] == 0 && !visited[nx][ny]) {
45                 visited[nx][ny] = true;
46                 q.push({nx, ny, cur.steps + 1});
47             }
48         }
49     }
50
51     return -1; // 无法到达
52 }
53
54 int main() {
55     // 0表示通路，1表示障碍
56     vector<vector<int>> maze = {
57         {0, 0, 0, 0},
58         {0, 1, 0, 0},
59         {0, 1, 0, 1},
60         {0, 0, 0, 0}
61     };
62
63     int steps = shortestPath(maze, 0, 0, 3, 3);
64     cout << "最短路径步数: " << steps << endl;

```

```

65         // 输出: 6
66
67         return 0;
68     }

```

Listing 5: BFS - 迷宫最短路径

4 DFS 与 BFS 的选择策略

如何选择 DFS 还是 BFS?

问题特征	推荐算法	原因
求最短路径/最少步数	BFS	逐层扩展，首次到达即最短
求所有可能解/路径	DFS	回溯机制天然适合枚举
树/图很深	BFS	避免递归栈溢出
树/图很宽	DFS	BFS 队列会占用大量空间
检测连通性	都可以	性能相近
拓扑排序	DFS	容易实现
状态空间很大	DFS+ 剪枝	内存占用小

5 经典例题详解

5.1 例题 1：岛屿数量（基础 DFS/BFS）

题目描述：给定一个由'1'（陆地）和'0'（水）组成的二维网格，计算网格中岛屿的数量。岛屿总是被水包围，并且每座岛屿只能由水平方向或竖直方向上相邻的陆地连接形成。

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     // DFS 淹没岛屿
6     void dfs(vector<vector<char>>& grid, int i, int j) {
7         int n = grid.size();
8         int m = grid[0].size();
9
10        // 边界检查

```

```

11         if (i < 0 || i >= n || j < 0 || j >= m || grid[i][j] == '
12             0') {
13                 return;
14     }
15
16     // 将当前陆地标记为水（淹没）
17     grid[i][j] = '0';
18
19     // 向四个方向扩展
20     dfs(grid, i - 1, j); // 上
21     dfs(grid, i + 1, j); // 下
22     dfs(grid, i, j - 1); // 左
23     dfs(grid, i, j + 1); // 右
24 }
25
26 int numIslands(vector<vector<char>>& grid) {
27     if (grid.empty()) return 0;
28
29     int n = grid.size();
30     int m = grid[0].size();
31     int count = 0;
32
33     for (int i = 0; i < n; i++) {
34         for (int j = 0; j < m; j++) {
35             if (grid[i][j] == '1') {
36                 count++; // 发现新岛屿
37                 dfs(grid, i, j); // 淹没整个岛屿
38             }
39         }
40     }
41
42     return count;
43 }
44
45 int main() {
46     vector<vector<char>> grid = {
47         {'1', '1', '0', '0', '0'},
48         {'1', '1', '0', '0', '0'},
49         {'0', '0', '1', '0', '0'},
50         {'0', '0', '0', '1', '1'}

```

```

50     };
51
52     cout << "岛屿数量: " << numIslands(grid) << endl;
53     // 输出: 3
54
55     return 0;
56 }

```

Listing 6: 例题 1: 岛屿数量 - DFS 解法

5.2 例题 2: 岛屿数量 - BFS 解法

```

1     #include <iostream>
2     #include <vector>
3     #include <queue>
4     using namespace std;
5
6     int numIslandsBFS(vector<vector<char>>& grid) {
7         if (grid.empty()) return 0;
8
9         int n = grid.size();
10        int m = grid[0].size();
11        int count = 0;
12
13        int dx[] = {-1, 1, 0, 0};
14        int dy[] = {0, 0, -1, 1};
15
16        for (int i = 0; i < n; i++) {
17            for (int j = 0; j < m; j++) {
18                if (grid[i][j] == '1') {
19                    count++;
20
21                    // BFS 淹没岛屿
22                    queue<pair<int, int>> q;
23                    q.push({i, j});
24                    grid[i][j] = '0';
25
26                    while (!q.empty()) {
27                        auto [x, y] = q.front();
28                        q.pop();

```

```

29
30         for (int k = 0; k < 4; k++) {
31             int nx = x + dx[k];
32             int ny = y + dy[k];
33
34             if (nx >= 0 && nx < n && ny >= 0 &&
35                 ny < m
36                 && grid[nx][ny] == '1') {
37                 grid[nx][ny] = '0';
38                 q.push({nx, ny});
39             }
40         }
41     }
42 }
43
44
45     return count;
46 }

```

Listing 7: 例题 1 变体：岛屿数量 - BFS 解法

5.3 例题 3：单词接龙（BFS 求最短路径）

题目描述：给定两个单词（beginWord 和 endWord）和一个字典，找到从 beginWord 到 endWord 的最短转换序列的长度。每次转换只能改变一个字母，且转换后的单词必须在字典中。

```

1      #include <iostream>
2      #include <vector>
3      #include <queue>
4      #include <unordered_set>
5      #include <unordered_map>
6      using namespace std;
7
8      int ladderLength(string beginWord, string endWord,
9      vector<string>& wordList) {
10         // 将字典转换为集合，方便查找
11         unordered_set<string> wordSet(wordList.begin(), wordList.
12             end());

```



```

13 // 如果终点不在字典中，无法转换
14 if (wordSet.find(endWord) == wordSet.end()) {
15     return 0;
16 }
17
18 // BFS
19 queue<string> q;
20 q.push(beginWord);
21
22 int steps = 1; // 包含起始单词
23
24 while (!q.empty()) {
25     int size = q.size();
26
27     // 处理当前层的所有节点
28     for (int i = 0; i < size; i++) {
29         string word = q.front();
30         q.pop();
31
32         // 尝试改变每个位置的字母
33         for (int j = 0; j < word.length(); j++) {
34             char original = word[j];
35
36             // 尝试26个字母
37             for (char c = 'a'; c <= 'z'; c++) {
38                 word[j] = c;
39
40                 // 找到终点
41                 if (word == endWord) {
42                     return steps + 1;
43                 }
44
45                 // 如果在字典中，加入队列继续搜索
46                 if (wordSet.find(word) != wordSet.end()) {
47                     q.push(word);
48                     wordSet.erase(word); // 删除防止重复访问
49                 }
50             }
51         }
52     }
53 }

```

```

51
52         // 恢复原字母
53         word[j] = original;
54     }
55 }
56
57     steps++; // 层数增加
58 }
59
60     return 0; // 无法转换
61 }
62
63 int main() {
64     string beginWord = "hit";
65     string endWord = "cog";
66     vector<string> wordList = {"hot", "dot", "dog", "lot", "log", "cog"};
67
68     int result = ladderLength(beginWord, endWord, wordList);
69     cout << "最短转换序列长度: " << result << endl;
70     // 输出: 5 (hit -> hot -> dot -> dog -> cog)
71
72     return 0;
73 }

```

Listing 8: 例题 3: 单词接龙 - BFS

5.4 例题 4: 课程表（拓扑排序 - DFS）

题目描述: 现在你总共有 n 门课需要选，记为 0 到 $n-1$ 。在选修某些课程之前需要先修其他课程。给定课程总量和先决条件，判断是否可能完成所有课程的学习。

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     // 课程表问题 - DFS检测环
6     bool hasCycle(int course, vector<vector<int>>& graph,
7     vector<int>& visited) {
8         // 1: 正在访问（在当前DFS路径中）
9         // 2: 已经访问完成

```

```

10     if (visited[course] == 1) return true;    // 发现环
11     if (visited[course] == 2) return false;  // 已经处理过
12
13     visited[course] = 1;    // 标记为正在访问
14
15     // 访问所有后继课程
16     for (int next : graph[course]) {
17         if (hasCycle(next, graph, visited)) {
18             return true;
19         }
20     }
21
22     visited[course] = 2;    // 标记为访问完成
23     return false;
24 }
25
26 bool canFinish(int numCourses, vector<vector<int>>&
27 prerequisites) {
28     // 构建图
29     vector<vector<int>> graph(numCourses);
30     for (auto& pre : prerequisites) {
31         graph[pre[1]].push_back(pre[0]);
32     }
33
34     // 访问状态：0=未访问，1=访问中，2=已访问
35     vector<int> visited(numCourses, 0);
36
37     // 对每个节点进行DFS
38     for (int i = 0; i < numCourses; i++) {
39         if (visited[i] == 0) {
40             if (hasCycle(i, graph, visited)) {
41                 return false;    // 有环，无法完成
42             }
43         }
44     }
45
46     return true;    // 无环，可以完成
47 }
48
49 int main() {

```

```

49     int n = 4;
50     vector<vector<int>> prerequisites = {{1, 0}, {2, 0}, {3,
51         1}, {3, 2}};
52
53     cout << "能否完成所有课程: "
54     << (canFinish(n, prerequisites) ? "是" : "否") << endl;
55     // 输出: 是
56
57     return 0;
58 }

```

Listing 9: 例题 4: 课程表 - DFS 检测环

5.5 例题 5: 课程表 - BFS 拓扑排序

```

1     #include <iostream>
2     #include <vector>
3     #include <queue>
4     using namespace std;
5
6     bool canFinishBFS(int numCourses, vector<vector<int>>&
7         prerequisites) {
8         // 构建图和入度数组
9         vector<vector<int>> graph(numCourses);
10        vector<int> inDegree(numCourses, 0);
11
12        for (auto& pre : prerequisites) {
13            graph[pre[1]].push_back(pre[0]);
14            inDegree[pre[0]]++;
15        }
16
17        // 将所有入度为0的节点入队
18        queue<int> q;
19        for (int i = 0; i < numCourses; i++) {
20            if (inDegree[i] == 0) {
21                q.push(i);
22            }
23        }
24
25        int count = 0; // 可以完成的课程数

```

```

25
26 // BFS 拓扑排序
27 while (!q.empty()) {
28     int course = q.front();
29     q.pop();
30     count++;
31
32     // 减少所有后继课程的入度
33     for (int next : graph[course]) {
34         inDegree[next]--;
35         if (inDegree[next] == 0) {
36             q.push(next);
37         }
38     }
39 }
40
41 return count == numCourses; // 所有课程都完成了
42 }

```

Listing 10: 例题 4 变体：课程表 - BFS 拓扑排序

6 进阶题目详解

6.1 进阶题 1：被围绕的区域（DFS/BFS）

题目描述：给定一个二维矩阵，包含 'X' 和 'O'。找到所有被 'X' 围绕的区域，并将这些区域里所有的 'O' 用 'X' 填充。边界上的 'O' 不会被填充。

解题思路：

- 核心思想：反过来思考，找到不被包围的 'O'（边界上的 'O' 及其连通的 'O'）
- 从边界开始 DFS/BFS，标记所有与边界连通的 'O'
- 未被标记的 'O' 就是被包围的，改为 'X'

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     void dfs(vector<vector<char>>& board, int i, int j) {
6         int n = board.size();

```

```

7      int m = board[0].size();
8
9      if (i < 0 || i >= n || j < 0 || j >= m || board[i][j] !=
      '0') {
10         return;
11     }
12
13     // 标记为临时字符，表示不被包围
14     board[i][j] = '#';
15
16     dfs(board, i - 1, j);
17     dfs(board, i + 1, j);
18     dfs(board, i, j - 1);
19     dfs(board, i, j + 1);
20 }
21
22 void solve(vector<vector<char>>& board) {
23     if (board.empty()) return;
24
25     int n = board.size();
26     int m = board[0].size();
27
28     // 处理四条边界
29     for (int i = 0; i < n; i++) {
30         dfs(board, i, 0);        // 左边界
31         dfs(board, i, m - 1);    // 右边界
32     }
33
34     for (int j = 0; j < m; j++) {
35         dfs(board, 0, j);        // 上边界
36         dfs(board, n - 1, j);    // 下边界
37     }
38
39     // 转换
40     for (int i = 0; i < n; i++) {
41         for (int j = 0; j < m; j++) {
42             if (board[i][j] == '0') {
43                 board[i][j] = 'X'; // 被包围的改为X
44             } else if (board[i][j] == '#') {
45                 board[i][j] = '0'; // 不被包围的恢复

```

```

46         }
47     }
48 }
49 }
50
51 int main() {
52     vector<vector<char>> board = {
53         {'X', 'X', 'X', 'X'},
54         {'X', 'O', 'O', 'X'},
55         {'X', 'X', 'O', 'X'},
56         {'X', 'O', 'X', 'X'}
57     };
58
59     solve(board);
60
61     cout << "处理后的矩阵:" << endl;
62     for (auto& row : board) {
63         for (char c : row) {
64             cout << c << " ";
65         }
66         cout << endl;
67     }
68
69     return 0;
70 }

```

Listing 11: 进阶题 1: 被围绕的区域

6.2 进阶题 2: 扫雷游戏 (BFS + 模拟)

题目描述: 给定一个代表游戏面板的二维字符矩阵，点击一个位置，根据扫雷规则揭示该位置。

解题思路:

- 如果点击到地雷 (M)，直接改为 X 并结束
- 如果点击到空格 (E)，用 BFS/DFS 展开：
 - 计算周围地雷数量
 - 如果有地雷，显示数字
 - 如果没地雷，显示 B，并递归展开周围 8 个格子

```

1      #include <iostream>
2      #include <vector>
3      #include <queue>
4      using namespace std;
5
6      vector<vector<char>> updateBoard(vector<vector<char>>& board,
7      vector<int>& click) {
8          int n = board.size();
9          int m = board[0].size();
10         int x = click[0], y = click[1];
11
12         // 点击到地雷
13         if (board[x][y] == 'M') {
14             board[x][y] = 'X';
15             return board;
16         }
17
18         // 8个方向
19         int dx[] = {-1, -1, -1, 0, 0, 1, 1, 1};
20         int dy[] = {-1, 0, 1, -1, 1, -1, 0, 1};
21
22         // BFS
23         queue<pair<int, int>> q;
24         q.push({x, y});
25
26         // 标记已处理，避免重复入队
27         vector<vector<bool>> inQueue(n, vector<bool>(m, false));
28         inQueue[x][y] = true;
29
30         while (!q.empty()) {
31             auto [cx, cy] = q.front();
32             q.pop();
33
34             // 计算周围地雷数量
35             int mines = 0;
36             vector<pair<int, int>> neighbors;
37
38             for (int i = 0; i < 8; i++) {
39                 int nx = cx + dx[i];

```



```

40         int ny = cy + dy[i];
41
42         if (nx >= 0 && nx < n && ny >= 0 && ny < m) {
43             if (board[nx][ny] == 'M') {
44                 mines++;
45             } else if (!inQueue[nx][ny]) {
46                 neighbors.push_back({nx, ny});
47             }
48         }
49     }
50
51     if (mines > 0) {
52         // 有地雷，显示数字
53         board[cx][cy] = '0' + mines;
54     } else {
55         // 没有地雷，递归展开
56         board[cx][cy] = 'B';
57         for (auto [nx, ny] : neighbors) {
58             q.push({nx, ny});
59             inQueue[nx][ny] = true;
60         }
61     }
62 }
63
64 return board;
65 }
66
67 int main() {
68     vector<vector<char>> board = {
69         {'E', 'E', 'E', 'E', 'E'},
70         {'E', 'E', 'M', 'E', 'E'},
71         {'E', 'E', 'E', 'E', 'E'},
72         {'E', 'E', 'E', 'E', 'E'}
73     };
74     vector<int> click = {3, 0};
75
76     auto result = updateBoard(board, click);
77
78     cout << "扫雷结果:" << endl;
79     for (auto& row : result) {

```

```

80         for (char c : row) {
81             cout << c << " ";
82         }
83         cout << endl;
84     }
85
86     return 0;
87 }

```

Listing 12: 进阶题 2: 扫雷游戏

6.3 进阶题 3: 太平洋大西洋水流问题

题目描述：给定一个 $m \times n$ 的非负整数矩阵表示地形高度。水可以从高处向低处流。求所有同时能流向太平洋和大西洋的坐标。太平洋在左边界和上边界，大西洋在右边界和下边界。

```

1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      void dfs(vector<vector<int>>& heights, vector<vector<bool>>&
6              ocean,
7              int i, int j, int prevHeight) {
8          int n = heights.size();
9          int m = heights[0].size();
10
11         // 检查边界和高度条件
12         if (i < 0 || i >= n || j < 0 || j >= m
13             || ocean[i][j] || heights[i][j] < prevHeight) {
14             return;
15         }
16
17         // 标记为可达
18         ocean[i][j] = true;
19
20         // 向四个方向流动（注意：水从低往高流是反过来的）
21         dfs(heights, ocean, i - 1, j, heights[i][j]);
22         dfs(heights, ocean, i + 1, j, heights[i][j]);
23         dfs(heights, ocean, i, j - 1, heights[i][j]);
24         dfs(heights, ocean, i, j + 1, heights[i][j]);

```

```

24     }
25
26     vector<vector<int>> pacificAtlantic(vector<vector<int>>&
27         heights) {
28         if (heights.empty()) return {};
29
30         int n = heights.size();
31         int m = heights[0].size();
32
33         vector<vector<bool>> pacific(n, vector<bool>(m, false));
34         vector<vector<bool>> atlantic(n, vector<bool>(m, false));
35
36         // 从太平洋边界开始DFS（左边界和上边界）
37         for (int i = 0; i < n; i++) {
38             dfs(heights, pacific, i, 0, INT_MIN);
39         }
40         for (int j = 0; j < m; j++) {
41             dfs(heights, pacific, 0, j, INT_MIN);
42         }
43
44         // 从大西洋边界开始DFS（右边界和下边界）
45         for (int i = 0; i < n; i++) {
46             dfs(heights, atlantic, i, m - 1, INT_MIN);
47         }
48         for (int j = 0; j < m; j++) {
49             dfs(heights, atlantic, n - 1, j, INT_MIN);
50         }
51
52         // 收集同时能流向两大洋的点
53         vector<vector<int>> result;
54         for (int i = 0; i < n; i++) {
55             for (int j = 0; j < m; j++) {
56                 if (pacific[i][j] && atlantic[i][j]) {
57                     result.push_back({i, j});
58                 }
59             }
60         }
61
62         return result;

```

```

63
64     int main() {
65         vector<vector<int>> heights = {
66             {1, 2, 2, 3, 5},
67             {3, 2, 3, 4, 4},
68             {2, 4, 5, 3, 1},
69             {6, 7, 1, 4, 5},
70             {5, 1, 1, 2, 4}
71         };
72
73         auto result = pacificAtlantic(heights);
74
75         cout << "同时能流向太平洋和大西洋的坐标:" << endl;
76         for (auto& pos : result) {
77             cout << "(" << pos[0] << "," << pos[1] << ") ";
78         }
79         cout << endl;
80
81         return 0;
82     }

```

Listing 13: 进阶题 3: 太平洋大西洋水流问题

6.4 进阶题 4: 最短路径访问所有节点（状态压缩 BFS）

题目描述：存在一个由 n 个节点组成的无向连通图，节点编号从 0 到 $n-1$ 。找出最短的路径，使得从任意节点出发，访问完所有节点。可以以任意节点作为起点，任意节点作为终点，可以多次访问节点和重复经过边。

```

1     #include <iostream>
2     #include <vector>
3     #include <queue>
4     #include <tuple>
5     using namespace std;
6
7     int shortestPathLength(vector<vector<int>>& graph) {
8         int n = graph.size();
9
10        // BFS队列：（当前节点，访问状态，路径长度）
11        queue<tuple<int, int, int>> q;
12

```

```

13 // visited[node][state] 防止重复访问
14 vector<vector<bool>> visited(n, vector<bool>(1 << n,
15     false));
16
17 // 可以从任意节点开始
18 for (int i = 0; i < n; i++) {
19     int state = 1 << i;
20     q.push({i, state, 0});
21     visited[i][state] = true;
22 }
23
24 int target = (1 << n) - 1; // 所有节点都访问过的状态
25
26 while (!q.empty()) {
27     auto [node, state, dist] = q.front();
28     q.pop();
29
30     // 所有节点都访问了
31     if (state == target) {
32         return dist;
33     }
34
35     // 访问相邻节点
36     for (int next : graph[node]) {
37         int newState = state | (1 << next);
38
39         if (!visited[next][newState]) {
40             visited[next][newState] = true;
41             q.push({next, newState, dist + 1});
42         }
43     }
44
45     return -1; // 不应该到达这里
46 }
47
48 int main() {
49     vector<vector<int>> graph = {
50         {1, 2, 3},
51         {0},

```

```

52         {0},
53         {0}
54     };
55
56     cout << "最短路径长度: " << shortestPathLength(graph) <<
57         endl;
58     // 输出: 4
59
60     graph = {{1}, {0, 2, 4}, {1, 3, 4}, {2}, {1, 2}};
61     cout << "最短路径长度: " << shortestPathLength(graph) <<
62         endl;
63     // 输出: 4
64
65     return 0;
66 }

```

Listing 14: 进阶题 4: 最短路径访问所有节点 - BFS+ 状态压缩

6.5 进阶题 5: 01 矩阵（多源 BFS）

题目描述：给定一个由 0 和 1 组成的矩阵，找出每个元素到最近的 0 的距离。

```

1     #include <iostream>
2     #include <vector>
3     #include <queue>
4     using namespace std;
5
6     vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
7         int n = mat.size();
8         int m = mat[0].size();
9
10        vector<vector<int>> dist(n, vector<int>(m, INT_MAX));
11        queue<pair<int, int>> q;
12
13        // 将所有0入队（多源BFS）
14        for (int i = 0; i < n; i++) {
15            for (int j = 0; j < m; j++) {
16                if (mat[i][j] == 0) {
17                    dist[i][j] = 0;
18                    q.push({i, j});
19                }

```

```

20         }
21     }
22
23     int dx[] = {-1, 1, 0, 0};
24     int dy[] = {0, 0, -1, 1};
25
26     // BFS
27     while (!q.empty()) {
28         auto [x, y] = q.front();
29         q.pop();
30
31         for (int k = 0; k < 4; k++) {
32             int nx = x + dx[k];
33             int ny = y + dy[k];
34
35             if (nx >= 0 && nx < n && ny >= 0 && ny < m
36                 && dist[nx][ny] > dist[x][y] + 1) {
37                 dist[nx][ny] = dist[x][y] + 1;
38                 q.push({nx, ny});
39             }
40         }
41     }
42
43     return dist;
44 }
45
46 int main() {
47     vector<vector<int>> mat = {
48         {0, 0, 0},
49         {0, 1, 0},
50         {1, 1, 1}
51     };
52
53     auto result = updateMatrix(mat);
54
55     cout << "到最近0的距离:" << endl;
56     for (auto& row : result) {
57         for (int d : row) {
58             cout << d << " ";
59         }

```

```

60         cout << endl;
61     }
62     // 输出:
63     // 0 0 0
64     // 0 1 0
65     // 1 2 1
66
67     return 0;
68 }

```

Listing 15: 进阶题 5: 01 矩阵 - 多源 BFS

7 搜索算法的优化技巧

7.1 剪枝优化

DFS 剪枝策略

1. 可行性剪枝: 当前状态不可能达到目标时提前返回
2. 最优性剪枝: 当前状态不如已知最优解时提前返回
3. 对称性剪枝: 利用问题的对称性减少搜索空间
4. 记忆化搜索: 避免重复搜索相同状态

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     vector<vector<string>> result;
6
7     bool isValid(vector<string>& board, int row, int col, int n)
8     {
9         // 检查列
10        for (int i = 0; i < row; i++) {
11            if (board[i][col] == 'Q') return false;
12        }
13
14        // 检查左上方对角线

```



```

14         for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--,
15             j--) {
16             if (board[i][j] == 'Q') return false;
17         }
18         // 检查右上方对角线
19         for (int i = row - 1, j = col + 1; i >= 0 && j < n; i--,
20             j++) {
21             if (board[i][j] == 'Q') return false;
22         }
23         return true;
24     }
25
26     void backtrack(vector<string>& board, int row, int n) {
27         if (row == n) {
28             result.push_back(board);
29             return;
30         }
31
32         for (int col = 0; col < n; col++) {
33             // 可行性剪枝：只尝试合法位置
34             if (isValid(board, row, col, n)) {
35                 board[row][col] = 'Q';
36                 backtrack(board, row + 1, n);
37                 board[row][col] = '.';
38             }
39         }
40     }
41
42     vector<vector<string>> solveNQueens(int n) {
43         vector<string> board(n, string(n, '.'));
44         backtrack(board, 0, n);
45         return result;
46     }
47
48     int main() {
49         int n = 4;
50         auto solutions = solveNQueens(n);
51

```

```

52         cout << n << "皇后问题的解法数量: " << solutions.size()
53             << endl;
54         for (auto& solution : solutions) {
55             for (auto& row : solution) {
56                 cout << row << endl;
57             }
58             cout << endl;
59         }
60         return 0;
61     }

```

Listing 16: DFS 剪枝示例: N 皇后问题

7.2 双向 BFS

```

1     #include <iostream>
2     #include <vector>
3     #include <unordered_set>
4     #include <unordered_map>
5     using namespace std;
6
7     int ladderLengthBiBFS(string beginWord, string endWord,
8         vector<string>& wordList) {
9         unordered_set<string> dict(wordList.begin(), wordList.end
10             ());
11         if (!dict.count(endWord)) return 0;
12
13         unordered_set<string> beginSet{beginWord};
14         unordered_set<string> endSet{endWord};
15         int steps = 1;
16
17         while (!beginSet.empty() && !endSet.empty()) {
18             // 总是扩展较小的集合
19             if (beginSet.size() > endSet.size()) {
20                 swap(beginSet, endSet);
21             }
22
23             unordered_set<string> nextSet;

```

```

24         for (string word : beginSet) {
25             for (int i = 0; i < word.length(); i++) {
26                 char original = word[i];
27
28                 for (char c = 'a'; c <= 'z'; c++) {
29                     word[i] = c;
30
31                     if (endSet.count(word)) {
32                         return steps + 1;
33                     }
34
35                     if (dict.count(word)) {
36                         nextSet.insert(word);
37                         dict.erase(word);
38                     }
39                 }
40
41                 word[i] = original;
42             }
43         }
44
45         beginSet = nextSet;
46         steps++;
47     }
48
49     return 0;
50 }

```

Listing 17: 双向 BFS 示例：单词接龙优化

7.3 记忆化搜索 (Memoization)

```

1     #include <iostream>
2     #include <vector>
3     #include <algorithm>
4     using namespace std;
5
6     int dfs(vector<vector<int>>& matrix, vector<vector<int>>&
7         memo,
8         int i, int j) {

```

```

8         if (memo[i][j] != 0) {
9             return memo[i][j]; // 记忆化：直接返回已计算的结果
10        }
11
12        int n = matrix.size();
13        int m = matrix[0].size();
14        int maxLen = 1;
15
16        int dx[] = {-1, 1, 0, 0};
17        int dy[] = {0, 0, -1, 1};
18
19        for (int k = 0; k < 4; k++) {
20            int ni = i + dx[k];
21            int nj = j + dy[k];
22
23            if (ni >= 0 && ni < n && nj >= 0 && nj < m
24                && matrix[ni][nj] > matrix[i][j]) {
25                maxLen = max(maxLen, 1 + dfs(matrix, memo, ni, nj
26                    ));
27            }
28        }
29
30        memo[i][j] = maxLen; // 存储结果
31        return maxLen;
32    }
33
34    int longestIncreasingPath(vector<vector<int>>& matrix) {
35        if (matrix.empty()) return 0;
36
37        int n = matrix.size(), m = matrix[0].size();
38        vector<vector<int>> memo(n, vector<int>(m, 0));
39        int result = 0;
40
41        for (int i = 0; i < n; i++) {
42            for (int j = 0; j < m; j++) {
43                result = max(result, dfs(matrix, memo, i, j));
44            }
45        }
46
47        return result;

```

```

47     }
48
49     int main() {
50         vector<vector<int>> matrix = {
51             {9, 9, 4},
52             {6, 6, 8},
53             {2, 1, 1}
54         };
55
56         cout << "最长递增路径长度: " << longestIncreasingPath(
57             matrix) << endl;
58         // 输出: 4 (1 -> 2 -> 6 -> 9)
59
60         return 0;
61     }

```

Listing 18: 记忆化搜索示例：最长递增路径

8 总结与对比

BFS vs DFS 核心对比总结

特性	BFS	DFS
数据结构	队列	栈（或递归）
遍历顺序	层次遍历	深度遍历
最短路径	能找到最优解	不一定
空间复杂度	$O(w)$ - 宽度	$O(h)$ - 深度
实现难度	较简单	递归简单，迭代较难
适用场景	最短路、层序	枚举所有解、回溯
栈溢出风险	无（迭代）	有（递归深度大）
剪枝难度	较难	容易

面试高频考点

1. 岛屿系列：岛屿数量、最大岛屿面积、岛屿周长
2. 迷宫系列：最短路径、墙与门、逃离迷宫
3. 单词系列：单词接龙、单词搜索
4. 排列组合：全排列、子集、组合总和
5. 图遍历：克隆图、课程表、网络延迟
6. 矩阵搜索：被围绕区域、太平洋大西洋水流
7. 状态搜索：打开转盘锁、滑动谜题

9 练习题推荐

1. DFS 基础：

- 岛屿的最大面积（LeetCode 695）
- 省份数量（LeetCode 547）
- 组合总和（LeetCode 39）

2. BFS 基础：

- 二叉树的层序遍历（LeetCode 102）
- 墙与门（LeetCode 286）
- 腐烂的橘子（LeetCode 994）

3. 进阶挑战：

- 打开转盘锁（LeetCode 752）
- 滑动谜题（LeetCode 773）
- 公交路线（LeetCode 815）

4. 困难模式：

- 最短超级串（LeetCode 943）
- 逃离大迷宫（LeetCode 1036）
- 访问所有节点的最短路径（LeetCode 847）